OpenCV: Caras de confianza

Sistema de Reconocimiento Facial, Liveness Detection y Control de Acceso

Autores: Ismael Miguel Martín / Javier Marcos Tobar

Fecha de creación: 8 de noviembre de 2026

Índice de contenidos

1. Funciones principales del proyecto y su relación con OpenCV

- Módulo de Procesado Facial y Reconocimiento (src/vision.py)
- Módulo de Prueba de Vida y Gestos (src/gestos.py)
- Módulo de Control y Flujo de Sesión (src/autenticador.py y src/creador_perfiles.py)
- Módulo de Depuración Visual y Telemetría (src/depurador.py)
- Módulo de Gestión de Credenciales y Estadísticas (src/usuarios.py)
- Módulo de Interfaz Visual sobre Cámara (src/interfaz.py)

2. Clasificación de funciones de OpenCV

- 2.1. Funciones vistas en las sesiones de clase
- 2.2. Funciones no vistas en clase: justificación y necesidad frente a lo visto en clase

3. Síntesis de integración

4. Bibliografía y referencias

Este documento recoge el análisis funcional del sistema de autenticación biométrica, destacando las funciones principales implementadas en el código fuente y su relación directa con las operaciones de visión por computador proporcionadas por la biblioteca OpenCV. Se expone qué cometido cumple cada función dentro de la arquitectura modular del proyecto, qué llamadas de OpenCV utiliza para su ejecución, y se presenta una síntesis comparativa entre los métodos vistos en las sesiones prácticas de la asignatura y las funciones complementarias avanzadas incorporadas.

1. Funciones principales del proyecto y su relación con OpenCV

A continuación, se detallan las funciones más representativas del proyecto organizadas según su área de responsabilidad dentro del sistema:

Módulo de Procesado Facial y Reconocimiento (src/vision.py)

- **detectar_caras(frame, clasificador=None)**

Propósito: Localiza rostros humanos en el fotograma actual tras normalizar la iluminación de la escena y descarta falsos positivos por debajo del área mínima.

Relación con OpenCV: Aplica compensación previa de iluminación mediante **ajustar_brillo_histograma**, calculando el histograma en escala de grises con **cv2.calcHist** y ajustando el canal V en el espacio de color HSV con **cv2.cvtColor**, **cv2.split** y **cv2.merge**. Convierte la imagen a escala de grises y ejecuta la detección multiescala mediante **cv2.CascadeClassifier.detectMultiScale(...)** para obtener las coordenadas envolventes (x, y, w, h). Finalmente filtra detecciones descartando las cajas cuyo producto ancho por alto sea inferior a **AREA_MINIMA_CARA** (8.000 píxeles).

- **alinear_ojos(cara)**

Propósito: Corrige posibles ladeos de cabeza para normalizar la orientación del rostro antes de la comparación biométrica.

Relación con OpenCV: Detecta la posición de ambos ojos en el hemisferio facial superior mediante clasificador Haar ocular (**haarcascade_eye.xml**). Calcula el ángulo de inclinación entre pupilas mediante **np.arctan2** y obtiene la matriz afín bidimensional de rotación con **cv2.getRotationMatrix2D**. Aplica la transformación rotacional sobre el fotograma con **cv2.warpAffine** empleando interpolación bilineal y relleno de bordes con **cv2.BORDER_REPLICATE** para nivelar horizontalmente la mirada.

- **normalizar_cara(gris, caja=None)**

Propósito: Homogeneiza las dimensiones geométricas, nitidez y distribución lumínica de la cara recortada a resolución estándar fija de 120x120 píxeles.

Relación con OpenCV: Si la alineación ocular está habilitada, nivela la rotación de pose con **alinear_ojos**. Descarta un 8% perimetral de bordes (**MARGEN_RECORTE_CARA = 0.08**) para eliminar fondos no faciales. Redimensiona el recorte al tamaño canónico de 120x120 píxeles (**TAM_CARA**) con **cv2.resize** empleando interpolación de área **cv2.INTER_AREA**. Equilibra el contraste local mediante ecualización adaptativa con **cv2.createCLAHE** (clipLimit=2.0, tileGridSize=(4, 4)) y atenúa ruido aleatorio de alta frecuencia con filtro Gaussiano **cv2.GaussianBlur** (3x3).

- **verificar_rostro_estatico(caja_actual, caja_previa, frame_gris, frames_estables=0)**

Propósito: Determina si un rostro detectado se mantiene centrado, frontal y quieto antes de avanzar la calibración en modo guardia o en retorno de ausencia.

Relación con OpenCV: Verifica la relación de aspecto frontal (ancho/alto entre 0.70 y 1.30) y evalúa el desplazamiento euclídeo del centroide respecto al fotograma previo con **np.hypot**. Comprueba la ausencia de movimiento aplicando sustracción adaptativa de fondo con **cv2.createBackgroundSubtractorKNN** (history=15, dist2Threshold=400.0, detectShadows=False), extrae la máscara de primer plano en la región facial, segmenta contornos de movimiento mediante **cv2.findContours(..., cv2.RETR_LIST, cv2.CHAIN_APPROX_NONE)** y calcula el área máxima con **abs(cv2.contourArea(contorno_max, True))**. Si el área de movimiento supera el umbral **UMBRAL_MOVIMIENTO_KNN** (600 px), penaliza la estabilidad (-2 frames); si permanece quieto, acumula fotogramas de fijación (+1 frame).

- **identificar_rostro(cara_detectada, lista_nombres, lista_caras) / verificar_usuario_especifico(...)**

Propósito: Evalúa el grado de similitud biométrica entre el rostro normalizado en vivo y las plantillas de referencia autorizadas registradas.

Relación con OpenCV: Aplica comparación matricial mediante **cv2.matchTemplate** empleando el criterio de correlación normalizada de diferencias cuadráticas **cv2.TM_SQDIFF_NORMED**, y localiza el valor mínimo de discrepancia con **cv2.minMaxLoc**. Convierte la distancia en porcentaje de similitud biométrica mediante la fórmula (1.0 - score) * 100.0 y valida si el valor supera el umbral de coincidencia predefinido.

Módulo de Prueba de Vida y Gestos (src/gestos.py)

- **extraer_rois(frame, x, y, w, h, angulo, offsets_ojos)**

Propósito: Aísla las regiones faciales clave (ojo izquierdo, ojo derecho, cejas y boca) considerando el ángulo de inclinación de pose.

Relación con OpenCV: Utiliza **cv2.getRotationMatrix2D** y **cv2.warpAffine** para generar un fotograma orientado donde las subregiones anatómicas se recortan de forma precisa sin desajustes debidos a la inclinación de la cabeza.

- **_evaluar_sonrisa(rois), _evaluar_ojos(rois) y _medir_distancia_cejas(rois)**

Propósito: Valida gestos activos requeridos al usuario (sonrisa mostrando dientes, guiño de ojo izquierdo/derecho o elevación de cejas) dentro de la clase EvaluadorGestos.

Relación con OpenCV: En la boca, normaliza a 100x60 px (**TAM_BOCA_CANONICA**) con **cv2.resize**, aplica discriminación colorimétrica de esmalte dental en espacio BGR (balanceando cotas de luminosidad y diferencias cromáticas |R - B| y |R - G|), limpia la máscara con **cv2.getStructuringElement** y **cv2.morphologyEx** (aperturas y cierres morfológicos), extrae las formas con **cv2.findContours** y evalúa dimensiones con **cv2.boundingRect**. En los ojos, calcula la relación de aspecto vertical/horizontal amortiguada en una ventana temporal deslizante y aplica la regla K de apertura ocular relativa (**FACTOR_K_CIERRE_OJO = 0.78**) frente al estado neutro. En las cejas, mide la distancia vertical relativa normalizada respecto a la distancia interpupilar.

Módulo de Control y Flujo de Sesión (src/autenticador.py y src/creador_perfiles.py)

- **autenticar_usuario(camara, clasificador)**

Propósito: Coordina la máquina de estados completa del sistema: vigilancia en guardia, autenticación en pantalla, reto facial de prueba de vida, sesión activa, guardia de ausencia temporal (10s) con retorno anti-usurpación y bloqueo de equipo.

Relación con OpenCV: Captura secuencialmente los fotogramas de la webcam mediante **cv2.VideoCapture.read()**, aplica efecto espejo con **cv2.flip(frame, 1)**, gestiona eventos de teclado con **cv2.waitKey(1)** y refresca la visualización con **cv2.imshow**. Integra la sustracción de fondo KNN para estabilizar el rostro, normaliza recortes antes de la autenticación y salvaguarda evidencias fotográficas en disco con **cv2.imwrite** ante accesos no autorizados o intentos de usurpación de sesión.

- **capturar_poses_usuario(nombre_usuario, rol, password)**

Propósito: Guía el proceso interactivo de registro tomando las muestras fotográficas de perfil del nuevo usuario.

Relación con OpenCV: Captura la imagen estabilizada a través de **cv2.VideoCapture**, renderiza instrucciones visuales en vivo y guarda las plantillas normalizadas (120x120 px) en disco utilizando **cv2.imwrite**.

Módulo de Depuración Visual y Telemetría (src/depurador.py)

- **mostrar_debug_estabilidad(...), mostrar_debug_matching(...), mostrar_debug_normalizacion(...)**

Propósito: Proporciona un entorno interactivo de inspección visual en tiempo real con 6 ventanas auxiliares según la fase operativa activa, permitiendo pausar el flujo paso a paso con la tecla [S].

Relación con OpenCV: Genera mosaicos visuales concatenando matrices con **np.hstack**, transforma matrices de discrepancia en mapas térmicos perceptivos aplicando **cv2.applyColorMap(..., cv2.COLORMAP_JET)**, dibuja y resalta contornos de movimiento en la máscara KNN con **cv2.drawContours**, fusiona paneles de telemetría translúcidos con **cv2.addWeighted**, y gestiona el ciclo de vida de ventanas de inspección con **cv2.namedWindow** y **cv2.destroyWindow** sincronizado con **cv2.waitKey(1)**.

Módulo de Gestión de Credenciales y Estadísticas (src/usuarios.py)

- **registrar_inicio_sesion(...), registrar_fin_sesion(...), registrar_evento_seguridad(...)**

Propósito: Administra la persistencia de credenciales, roles y estadísticas biométricas y de seguridad en usuarios.json.

Relación con OpenCV: Audita eventos de seguridad asociados a la visión por computador: cuenta evidencias fotográficas de intrusión en disco (**contar_intrusos_desde**), contabiliza intentos de usurpación neutralizados durante ausencias, registra retornos validados exitosamente y consolida en disco el promedio de similitud facial continua evaluado cada 15 fotogramas durante la sesión activa.

Módulo de Interfaz Visual sobre Cámara (src/interfaz.py)

- **dibujar_hud_sesion(...) y menús interactivos modales**

Propósito: Renderiza tarjetas de estado en vivo, panel desplegable de estadísticas [m], opciones de rol [o], barras dinámicas de calibración/ausencia y diálogos modales interactivos superpuestos sobre la cámara.

Relación con OpenCV: Dibuja recuadros, divisiones y barras de progreso con **cv2.rectangle** y **cv2.line**, imprime textos informativos y métricas de telemetría con **cv2.putText**, y aplica combinación lineal ponderada mediante **cv2.addWeighted** para producir paneles translúcidos que no obstruyen la visualización del usuario.

2. Clasificación de funciones de OpenCV

2.1. Funciones vistas en las sesiones de clase

A continuación se relacionan las funciones estudiadas a lo largo de las sesiones prácticas de la asignatura que han servido como base de implementación:

- **cv2.VideoCapture (read, isOpened, release):** Apertura, lectura secuencial de fotogramas y cierre del dispositivo de cámara web.
- **cv2.CascadeClassifier y detectMultiScale:** Carga de clasificadores Haar Cascade para detección multiescala de rostros y ojos.
- **cv2.cvtColor:** Transformación entre espacios de color (BGR a escala de grises y a espacio HSV).
- **cv2.matchTemplate y cv2.minMaxLoc:** Cálculo de similitud por correlación de matrices (TM_SQDIFF_NORMED) y obtención del valor mínimo de discrepancia.
- **cv2.createBackgroundSubtractorKNN:** Sustracción adaptativa de fondo basada en K vecinos más próximos para la detección de movimiento y verificación de quietud frontal del usuario.
- **cv2.findContours, cv2.boundingRect y cv2.contourArea:** Segmentación de contornos, cálculo de cajas envolventes y medición de superficies de movimiento y rasgos faciales.
- **cv2.drawContours:** Trazado y resaltado vectorial de contornos sobre fotogramas y máscaras binarias de movimiento.
- **cv2.imshow, cv2.waitKey y cv2.destroyAllWindows / cv2.destroyWindow:** Creación de ventanas, refresco interactivo, gestión de eventos de teclado y liberación de recursos.
- **cv2.rectangle, cv2.line y cv2.putText:** Trazado de figuras geométricas, barras de estado/progreso y textos informativos del HUD.
- **cv2.imread y cv2.imwrite:** Lectura y escritura de imágenes en disco para el almacenamiento de plantillas de confianza y evidencias fotográficas de intrusión.
- **cv2.resize:** Redimensionado de imágenes para estandarizar resoluciones de trabajo (TAM_CARA = 120x120 e invarianza de escala bucal).
- **cv2.split, cv2.merge y cv2.calcHist:** Separación y combinación de canales cromáticos y análisis del histograma de brillo para normalización lumínica adaptativa.
- **cv2.addWeighted:** Combinación lineal ponderada de matrices para producir paneles y fondos translúcidos sobre la imagen en vivo.

2.2. Funciones no vistas en clase: justificación y necesidad frente a lo visto en clase

Para solucionar limitaciones concretas de las técnicas estudiadas en las prácticas, se incorporaron las siguientes funciones avanzadas de OpenCV. A continuación se detalla su funcionamiento y el motivo por el cual resultaron necesarias frente a las alternativas vistas en clase:

- **cv2.createCLAHE (Contrast Limited Adaptive Histogram Equalization)**

Funcionamiento: Genera un ecualizador de histograma adaptativo que fragmenta la imagen en pequeñas rejillas locales (tiles de 4x4) y redistribuye las intensidades en cada una de ellas de forma independiente, limitando el contraste máximo (clipLimit=2.0) para evitar amplificar el ruido de fondo.

Necesidad: En las prácticas se estudió el análisis de histograma (**cv2.calcHist**) y ajustes globales de brillo y contraste. Sin embargo, si la iluminación de la estancia cambia entre el registro del usuario y la autenticación posterior (por ejemplo, por una ventana lateral o una bombilla), un ajuste global sobreexpone las zonas claras o deja a oscuras las sombras de la cara. CLAHE fue necesario frente a los ajustes globales porque equilibra las sombras faciales de forma local y uniforme sin quemar las partes iluminadas ni disparar el grano en las zonas oscuras.

- **cv2.getRotationMatrix2D y cv2.warpAffine**

Funcionamiento: cv2.getRotationMatrix2D calcula la matriz afín bidimensional a partir del centro de giro, el ángulo y el factor de escala. **cv2.warpAffine** aplica dicha transformación matemática proyectando y rotando los píxeles de la imagen con interpolación continua.

Necesidad: En clase, el recorte de regiones de interés (rostros u ojos) se realizaba únicamente mediante cajas rectangulares ortogonales fijas a los ejes horizontal y vertical (mediante coordenadas de recorte directo **img[y:y+h, x:x+w]** o **cv2.boundingRect**). Si el usuario inclina levemente la cabeza, el recorte ortogonal extrae los ojos y la boca con una orientación oblicua que no coincide con la plantilla de referencia frontal, provocando que la comparación matricial (**cv2.matchTemplate**) falle. El empleo de estas funciones fue indispensable para alinear horizontalmente la línea de los ojos antes de recortar, haciendo el reconocimiento robusto ante variaciones naturales de pose.

- **cv2.applyColorMap**

Funcionamiento: Aplica mapas de falso color predefinidos (como **cv2.COLORMAP_JET**) transformando matrices escalares unidimensionales en representaciones cromáticas de tres canales mediante gradientes térmicos (azul-cian-amarillo-rojo).

Necesidad: En clase, la visualización de matrices de discrepancia y respuestas de filtros se limitaba a representaciones monocromáticas en escala de grises cruda (**cv2.imshow**), donde el sistema visual humano apenas distingue pequeñas diferencias de brillo entre tonos oscuros vecinos.
cv2.applyColorMap fue necesario en el modo depurador visual para generar mapas de calor perceptivos en Template Matching, permitiendo diagnosticar de manera inmediata e intuitiva en qué regiones anatómicas concretas (ojos, nariz, pómulos) se concentran los errores de coincidencia con un contraste perceptivo muy superior.

- **cv2.GaussianBlur**

Funcionamiento: Aplica un filtro de convolución espacial basado en una campana de Gauss simétrica, otorgando un peso significativamente mayor al píxel central que a los píxeles periféricos de la ventana.

Necesidad: En clase se empleó **cv2.blur** (filtro de media simple o caja), el cual promedia todos los píxeles vecinos con el mismo peso uniforme. Este promedio indiscriminado difumina y degrada los bordes finos del rostro (como las comisuras labiales y el contorno ocular), que son esenciales para la comparación de plantillas. **cv2.GaussianBlur** fue necesario frente al filtro de media porque elimina el ruido aleatorio de alta frecuencia del sensor de la webcam preservando con mucha mayor fidelidad la estructura de bordes faciales.

- **cv2.morphologyEx y cv2.getStructuringElement**

Funcionamiento: cv2.getStructuringElement genera máscaras morfológicas con formas geométricas adaptadas (elipses o rectángulos). **cv2.morphologyEx** ejecuta operaciones combinadas de apertura (**MORPH_OPEN**: erosión seguida de dilatación) y cierre (**MORPH_CLOSE**: dilatación seguida de erosión).

Necesidad: En las prácticas se utilizaron operaciones aisladas de erosión (**cv2.erode**) y dilatación (**cv2.dilate**). En el análisis de gestos (detección de dientes para validar la sonrisa y apertura del párpado), una erosión simple reduce el tamaño real de la boca y una dilatación simple la agranda artificialmente. **cv2.morphologyEx** fue necesario frente a las operaciones individuales porque permite limpiar pequeños puntos de ruido disperso y cerrar discontinuidades en la máscara binaria sin alterar las dimensiones ni la geometría real de las facciones del usuario.

- **cv2.flip**

Funcionamiento: Invierte el orden de las columnas de la matriz del fotograma (flipCode=1), produciendo una reflexión especular horizontal.

Necesidad: En clase, el flujo capturado por **cv2.VideoCapture** se representaba tal cual en la ventana (**cv2.imshow**). Las webcams proporcionan por defecto una imagen directa, por lo que si el usuario se mueve a su derecha, en pantalla se observa un desplazamiento hacia la izquierda. Esto genera una sensación contraintuitiva y dificulta centrarse en la cámara. **cv2.flip** fue necesario para transformar la señal en un espejo natural, elemento fundamental para que el usuario pueda encuadrarse con facilidad dentro de las guías faciales e interactuar de forma intuitiva con el menú en pantalla.

3. Síntesis de integración

El diseño modular adoptado conecta las funciones específicas del dominio de la aplicación (captura de poses, verificación de usuario, inspección de gestos, modo depurador visual y renderizado del HUD) con las operaciones fundamentales de OpenCV. Las técnicas clásicas vistas en clase garantizan la captura continua, la localización de rasgos faciales, la sustracción adaptativa de fondo (KNN) para verificación de quietud y la interfaz en tiempo real, mientras que las operaciones avanzadas (CLAHE, alineación afín por vector ocular, mapas térmicos de falso color y filtros Gaussianos) aseguran la resistencia y robustez del sistema frente a perturbaciones del entorno físico como iluminaciones irregulares, ladeos de cabeza o ruidos de sensor.

4. Bibliografía y referencias

- OpenCV Development Team. (2024). *Open Source Computer Vision Library (OpenCV)*. Versión 4.x. Documentación oficial de referencia técnica y algorítmica. Disponible en: https://docs.opencv.org/
- Material docente de la asignatura. *Ejemplos y ejercicios propuestos durante las clases de Imagen Digital*. Guías prácticas de laboratorio y códigos de referencia desarrollados a lo largo del curso.